

QPU Static Circuit Compiler

User Guide

Bailey Forbes

June 8, 2026

1 Overview

The QPU Static Circuit Compiler is a browser-based TypeScript compiler that turns QPU protocol text into visual quantum circuits.

QPU Protocol $\rightarrow$ Parser $\rightarrow$ Compiler $\rightarrow$ Circuit Gates $\rightarrow$ React Renderer

The compiler recognizes quantum gates, derived logical gates, process commands, token management, checkpoint operations, and measurement operations.

2 Supported Operations

```
INCREASECYCLE COMPILEPROCESS FREE SET JOIN SPLIT  
CALL DECLARECHILD RUNCHILD RETURNVALS ACCEPTVALS  
MASTerval SAVE_STATE LOAD_STATE MAIN-PROCESS  
CREATETOKEN DELETETOKEN  
X H CNOT CCNOT PHASE  
NOT AND NAND OR XOR  
MEASURE
```

3 Quantum State Model

A qubit is a complex-valued state vector:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$$

where

$$|\alpha|^2 + |\beta|^2 = 1.$$

The values α and β are probability amplitudes. The probability of measuring 0 or 1 is:

$$P(0) = |\alpha|^2, \quad P(1) = |\beta|^2.$$

Measurement compresses a quantum state into a classical state:

$$\alpha |0\rangle + \beta |1\rangle \implies 0 \text{ or } 1.$$

In the React renderer, this should appear as a 3D qubit sphere collapsing into a 2D classical state card. Multiple qubits are combined with the tensor product:

$$|AB\rangle = |A\rangle \otimes |B\rangle.$$

This is the state-vector model behind JOIN.

4 Command Syntax

Most gates use `-I` for inputs and `-O` for outputs.

```
GATE -I input_tokens -O output_token
```

Examples:

```
X -I a:0 -O a:1
H -I q0:0 -O q0:1
CNOT -I control:0 -O target:1
CCNOT -I a:0 b:0 -O carry:1
PHASE=3.14159 -I q0:0 -O q0:1
```

Cycle suffixes are stripped for visual qubit mapping:

```
a:0
a:1
a:2
```

all map to the same token:

```
a
```

5 Primitive Quantum Gates

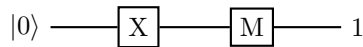
5.1 Pauli-X Gate and Classical NOT

The classical version is the NOT gate. The quantum version is the Pauli-X gate. Both flip a binary state, but the Pauli-X gate acts on quantum basis states and can also act linearly on superpositions.

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

$$X |0\rangle = |1\rangle, \quad X |1\rangle = |0\rangle.$$

```
MAIN-PROCESS XDemo
SET q0 0p
X -I q0 -O q0
MEASURE -I q0
```



Classical NOT and quantum Pauli-X comparison:

Input	Classical NOT	Pauli-X Output
0	1	$X 0\rangle = 1\rangle$
1	0	$X 1\rangle = 0\rangle$

5.2 Hadamard Gate and Classical Randomization

There is no exact classical equivalent to the Hadamard gate. A rough classical comparison is randomization, because measuring $H |0\rangle$ gives 0 or 1 with equal probability. The difference is that Hadamard creates amplitudes, not ordinary random bits.

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

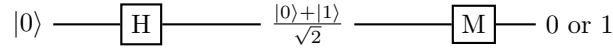
$$H |0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

$$H |1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$$

```

MAIN-PROCESS SuperpositionDemo
SET q0 0p
H -I q0 -0 q0
MEASURE -I q0

```



Classical randomization and quantum Hadamard comparison:

Input	Classical Random Bit	Hadamard Output
0	0 or 1	$H 0\rangle = \frac{ 0\rangle+ 1\rangle}{\sqrt{2}}$
1	0 or 1	$H 1\rangle = \frac{ 0\rangle- 1\rangle}{\sqrt{2}}$

The signs in the amplitudes matter. They allow interference, which ordinary classical randomness cannot represent.

5.3 Phase Gate and Classical Phase Absence

The phase gate has no direct classical bit equivalent. It changes the phase of the $|1\rangle$ component while leaving the measured basis value unchanged until interference or later gates expose the phase.

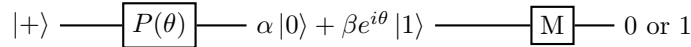
$$P(\theta) = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{bmatrix}$$

$$\alpha|0\rangle + \beta|1\rangle \mapsto \alpha|0\rangle + \beta e^{i\theta}|1\rangle.$$

```

MAIN-PROCESS PhaseDemo
SET q0 sp
PHASE=1.5708 -I q0 -0 q0
MEASURE -I q0

```



Reverse phase is supported by prefixing the gate with B:

```

BPHASE=1.5708 -I q0 -0 q0

```

$$BPHASE(\theta) = PHASE(-\theta).$$

Classical bit and quantum phase comparison:

Input	Classical Bit Operation	Phase Output
0	0	$P(\theta) 0\rangle = 0\rangle$
1	1	$P(\theta) 1\rangle = e^{i\theta} 1\rangle$

5.4 CNOT Gate and Classical XOR

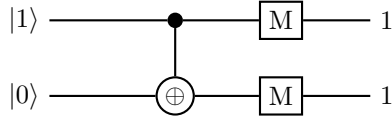
The classical comparison is XOR. The quantum version is CNOT, which is a reversible XOR because it preserves the control and writes the XOR result into the target.

$$target \leftarrow target \oplus control.$$

```

MAIN-PROCESS CnotDemo
SET control 1p
SET target 0p
CNOT -I control -0 target
MEASURE

```



Classical XOR and quantum CNOT comparison:

C	T	Classical $C \oplus T$	C'	T'
0	0	0	0	0
0	1	1	0	1
1	0	1	1	1
1	1	0	1	0

Quantum basis-state rule:

$$|C, T\rangle \mapsto |C, T \oplus C\rangle.$$

Input	CNOT Output
$ 00\rangle$	$ 00\rangle$
$ 01\rangle$	$ 01\rangle$
$ 10\rangle$	$ 11\rangle$
$ 11\rangle$	$ 10\rangle$

5.5 CCNOT / Toffoli Gate and Classical AND

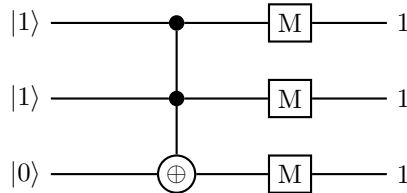
The classical comparison is AND. The quantum version is CCNOT, also called the Toffoli gate. It is a reversible AND because it preserves both inputs and writes the AND result into the target by XORing the target with $A \wedge B$.

$$target \leftarrow target \oplus (A \wedge B).$$

```

MAIN-PROCESS ToffoliDemo
SET A 1p
SET B 1p
SET target 0p
CCNOT -I A B -O target
MEASURE

```



Classical AND and quantum CCNOT comparison:

A	B	T	Classical $A \wedge B$	A'	B'	T'
0	0	0	0	0	0	0
0	0	1	0	0	0	1
0	1	0	0	0	1	0
0	1	1	0	0	1	1
1	0	0	0	1	0	0
1	0	1	0	1	0	1
1	1	0	1	1	1	1
1	1	1	1	1	1	0

Quantum basis-state rule:

$$|A, B, T\rangle \mapsto |A, B, T \oplus (A \wedge B)\rangle.$$

6 Derived Logical Gates

Derived gates behave like classical logic gates, but their quantum form must be reversible. Instead of destroying inputs, the compiler preserves inputs and writes the result into a target register.

6.1 AND

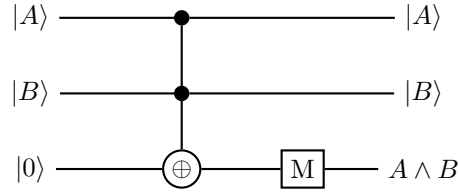
The classical version is AND. The quantum version is a CCNOT-based reversible AND that writes $A \wedge B$ into $0p$.

$$|A, B, 0\rangle \mapsto |A, B, A \wedge B\rangle.$$

$$0p \leftarrow 0p \oplus (A \wedge B).$$

Since $0p$ starts as 0, the final target is $A \wedge B$.

```
MAIN-PROCESS AndDemo
SET A 1p
SET B 1p
SET 0p 0p
CCNOT -I A B -O 0p
MEASURE -I 0p
```



Classical AND and quantum reversible AND comparison:

A	B	T	Classical $A \wedge B$	A'	B'	T'
0	0	0	0	0	0	0
0	1	0	0	0	1	0
1	0	0	0	1	0	0
1	1	0	1	1	1	1

6.2 OR

The classical version is OR. The quantum version is a reversible OR circuit that preserves A and B , then writes $A \vee B$ into $0p$.

The identity used is:

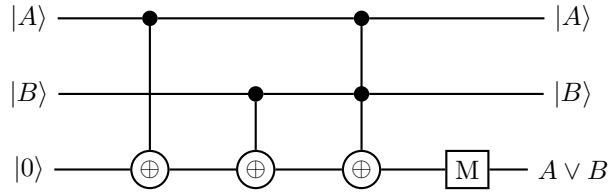
$$A \vee B = A \oplus B \oplus (A \wedge B).$$

Implementation:

$$0p \leftarrow 0p \oplus A \oplus B \oplus (A \wedge B).$$

Since $0p$ starts as 0, the final target is $A \vee B$.

```
MAIN-PROCESS OrDemo
SET A 1p
SET B 0p
SET 0p 0p
CNOT -I A -O 0p
CNOT -I B -O 0p
CCNOT -I A B -O 0p
MEASURE -I 0p
```



Classical OR and quantum reversible OR comparison:

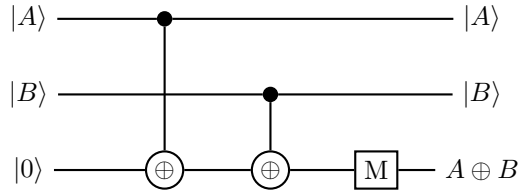
A	B	T	Classical $A \vee B$	A'	B'	T'
0	0	0	0	0	0	0
0	1	0	1	0	1	1
1	0	0	1	1	0	1
1	1	0	1	1	1	1

6.3 XOR

The classical version is XOR. The quantum version is a reversible XOR built from two CNOT operations. It preserves A and B , then writes $A \oplus B$ into $0p$.

$$|A, B, 0\rangle \mapsto |A, B, A \oplus B\rangle.$$

```
MAIN-PROCESS XorDemo
SET A 1p
SET B 0p
SET 0p 0p
CNOT -I A -0 0p
CNOT -I B -0 0p
MEASURE -I 0p
```



Classical XOR and quantum reversible XOR comparison:

A	B	T	Classical $A \oplus B$	A'	B'	T'
0	0	0	0	0	0	0
0	1	0	1	0	1	1
1	0	0	1	1	0	1
1	1	0	0	1	1	0

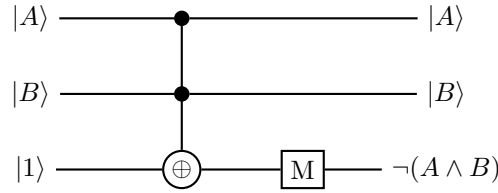
6.4 NAND

The classical version is NAND. The quantum version is a reversible NAND circuit that initializes the target to 1, then applies CCNOT.

$$target \leftarrow 1 \oplus (A \wedge B).$$

If the target begins as 1, CCNOT flips it to 0 only when both inputs are 1.

```
MAIN-PROCESS NandDemo
SET A 1p
SET B 1p
SET 1p 1p
CCNOT -I A B -0 1p
MEASURE -I 1p
```



Classical NAND and quantum reversible NAND comparison:

A	B	T	Classical $\neg(A \wedge B)$	A'	B'	T'
0	0	1	1	0	0	1
0	1	1	1	0	1	1
1	0	1	1	1	0	1
1	1	1	0	1	1	0

7 State and Memory Operations

7.1 SET

SET initializes or aliases a token.

```
SET q0 0p
SET q1 1p
SET q2 sp
SET alias q0
```

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

The `sp` token initializes a superposition-like state:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}.$$

7.2 JOIN

JOIN builds a larger register from smaller registers.

```
JOIN -I A:0 B:0 -O AB:1
```

$$|AB\rangle = |A\rangle \otimes |B\rangle.$$

7.3 SPLIT

SPLIT extracts a smaller register from a composite register.

```
SPLIT AB:1 A:2 2
```

A split is only physically clean when the composite state can be factored into independent pieces.

7.4 MEASURE

MEASURE collapses quantum amplitudes into classical bits.

```
MEASURE -I q0
MEASURE
```

$$\alpha |0\rangle + \beta |1\rangle \implies 0 \text{ or } 1.$$

With no `-I`, all known qubits are measured.

8 Process and Runtime Commands

These commands control compilation and process flow. They do not directly represent quantum matrices.

MAIN-PROCESS Defines the entry point.

COMPILEPROCESS Loads and compiles another process.

CALL Executes another process.

DECLARECHILD Declares a child process.

RUNCHILD Runs a child process.

RETURNVALS Returns values from a child.

ACCEPTVALS Captures child return values.

MASTERVAL Marks the final value of a process.

CREATETOKEN Creates logical registers.

DELETETOKEN Deletes logical registers.

FREE Marks memory entries as releasable.

SAVE_STATE Saves a checkpoint.

LOAD_STATE Restores a checkpoint.

INCREASECYCLE Advances the protocol clock.

9 Process Example

```
PARAMS: A:state B:state

MAIN-PROCESS ChildExample
SET result 0p
CCNOT -I A B -0 result
RETURNVALS result
```

A parent process can call a child process and accept its returned values:

```
MAIN-PROCESS ParentExample
SET A 1p
SET B 1p
CALL ChildExample -I A B
ACCEPTVALS output
MEASURE -I output
```

10 Compilation Output

The TypeScript compiler emits:

```
CompileResult {
  gates: CircuitGate[]
  qubitCount: number
  parsed: ParsedCommand[]
  log: string[]
  tokenMap: Record<string, number>
}
```

Each rendered gate contains:

```
id
type
step
targets
controls
phase
source
```

11 Visual Rendering Model

The React renderer should display:

- qubit lanes,
- draggable gate blocks,
- control dots for CNOT and CCNOT,
- target markers,
- measurement gates,
- 3D particle-style qubit states,
- collapsed 2D classical state cards after measurement.

The mobile-first flow is:

Gate Palette → Touch Drag → Circuit Lane → Simulation → Measurement Display

12 Static Deployment

The project is designed for static hosting.

React + TypeScript → npm build → GitHub Pages

No backend server is required for the MVP.

13 MVP Limitations

The current compiler parses or acknowledges some operations without fully lowering them into visual gates.

```
JOIN
SPLIT
FREE
SAVE_STATE
LOAD_STATE
MASTerval
COMPILEPROCESS
```

These operations preserve protocol compatibility and can be expanded into full runtime behavior later.